PyTorch and NN Programming Fundamentals

AI 500: Foundations of AI

Fall 2025

Contents

1 Introduction **3**

2 Tensor Fundamentals **3**
2.1 What are Tensors? . . . 3
2.2 Creating Tensors . . . 3
2.3 Tensor Operations . . . 3
2.3.1 Element-wise Operations . . . 3
2.3.2 Matrix Multiplication . . . 4
2.4 Shape Manipulation . . . 4
2.5 Practical Exercises: Mathematical Functions . . . 4
2.5.1 Function 1: Weighted Sum . . . 4
2.5.2 Function 2: Matrix-Vector Product with Element-wise Multiplication . . . 4
2.5.3 Function 3: Gram Matrix . . . 4
2.5.4 Function 4: Standardization . . . 5

3 PyTorch Ecosystem **5**
3.1 Devices and Data Types . . . 5
3.1.1 Devices . . . 5
3.1.2 Data Types . . . 5
3.2 Datasets and DataLoaders . . . 5
3.2.1 Dataset Class . . . 5
3.2.2 DataLoader . . . 5
3.3 Loss Functions . . . 6
3.4 Optimizers . . . 6
3.5 Training Loop Anatomy . . . 6
3.6 Model Persistence . . . 7

4 Building Neural Networks **7**
4.1 The `nn.Module` Class . . . 7
4.2 Common Layers . . . 7
4.3 Example Architecture . . . 7
4.4 Sequential Alternative . . . 8

5 Debugging and Best Practices **8**
5.1 Common Errors . . . 8
5.2 Debugging Checklist . . . 8
5.3 Best Practices . . . 8

6 Further Resources **8**
6.1 Official Documentation 8
6.2 Tutorials 9
6.3 Community and Practice 9
6.4 Advanced Topics 9

7 Conclusion **9**

1 Introduction

This tutorial provides a comprehensive introduction to PyTorch for neural network programming. By the end of this session, you will understand:

- Tensor operations and manipulation in PyTorch
- The PyTorch ecosystem: datasets, dataloaders, optimizers, and loss functions
- How to build custom neural networks using `nn.Module`

Prerequisites: Python programming, basic linear algebra, understanding of gradient descent and backpropagation.

2 Tensor Fundamentals

2.1 What are Tensors?

Tensors are multi-dimensional arrays that generalize scalars, vectors, and matrices to arbitrary dimensions. In PyTorch, tensors are the fundamental data structure:

- **0D tensor (scalar):** A single number, e.g., 5.0
- **1D tensor (vector):** $\mathbf{v} = [1, 2, 3]$
- **2D tensor (matrix):** $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$
- **3D+ tensors:** Used for batched data, images (batch, channels, height, width), sequences, etc.

PyTorch tensors support automatic differentiation (autograd), making them ideal for deep learning. Unlike NumPy arrays, tensors can be moved to GPUs for accelerated computation.

2.2 Creating Tensors

PyTorch provides several methods to create tensors:

```
import torch

# From Python lists
x = torch.tensor([1, 2, 3])
A = torch.tensor([[1, 2], [3, 4]])

# Using constructors
zeros = torch.zeros((3, 3))       # 3x3 matrix of zeros
ones = torch.ones((2, 4))         # 2x4 matrix of ones
rand = torch.rand((2, 2))         # Uniform random in [0, 1)
randn = torch.randn((2, 2))       # Standard normal distribution
arange = torch.arange(0, 10, 2)   # [0, 2, 4, 6, 8]
```

2.3 Tensor Operations

2.3.1 Element-wise Operations

Element-wise operations apply to corresponding elements:

$$\mathbf{C} = \mathbf{A} + \mathbf{B} \quad \Rightarrow \quad C_{ij} = A_{ij} + B_{ij} \tag{1}$$

$$\mathbf{C} = \mathbf{A} \odot \mathbf{B} \quad \Rightarrow \quad C_{ij} = A_{ij} \cdot B_{ij} \tag{2}$$

In PyTorch: `A + B`, `A - B`, `A * B`, `A / B`

2.3.2 Matrix Multiplication

For matrices $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$:

$$(\mathbf{AB})_{ij} = \sum_{k=1}^{n} A_{ik} B_{kj} \tag{3}$$

In PyTorch: `A @ B` (preferred over `torch.matmul(A, B)`)

2.4 Shape Manipulation

- `.shape`: Returns dimensions as a tuple
- `.reshape(new_shape)`: Returns a new tensor with specified shape
- `.view(new_shape)`: Similar to reshape but may share memory
- `.T`: Transpose for 2D tensors
- `.transpose(dim0, dim1)`: Swap two dimensions

Important: Total elements must remain constant during reshaping.

2.5 Practical Exercises: Mathematical Functions

Implement the following functions to practice tensor operations.

2.5.1 Function 1: Weighted Sum

Compute the weighted sum of a vector:

$$f_1(\mathbf{x}, \mathbf{w}) = \sum_{i=1}^{n} w_i x_i = \mathbf{w}^T \mathbf{x} \tag{4}$$

Constraints: $\mathbf{x}, \mathbf{w} \in \mathbb{R}^n$ (same shape). Return 0 if shapes mismatch.

2.5.2 Function 2: Matrix-Vector Product with Element-wise Multiplication

Given matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ and vector $\mathbf{v} \in \mathbb{R}^n$:

$$f_2(\mathbf{A}, \mathbf{v}) = (\mathbf{Av})^T \odot \mathbf{v}^T \tag{5}$$

For example, if:

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{pmatrix}, \quad \mathbf{v} = \begin{pmatrix} 1 \\ 1 \\ 1 \end{pmatrix}$$

Then $\mathbf{Av} = \begin{pmatrix} 6 \\ 15 \\ 24 \end{pmatrix}$, and the result is element-wise multiplication with $\mathbf{v}^T$.

Constraints: Check that $\mathbf{v}$ has shape compatible with $\mathbf{A}$.

2.5.3 Function 3: Gram Matrix

The Gram matrix captures correlations between features:

$$\mathbf{G} = \mathbf{A}^T \mathbf{A} \tag{6}$$

where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{G} \in \mathbb{R}^{n \times n}$.

Use case: Neural style transfer, feature correlation analysis.

2.5.4 Function 4: Standardization

Normalize a tensor along a dimension:

$$\hat{\mathbf{x}} = \frac{\mathbf{x} - \mu}{\sigma} \tag{7}$$

where $\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$ and $\sigma = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2}$.
Methods: Use `.mean()` and `.std()` along specified dimensions.

3 PyTorch Ecosystem

3.1 Devices and Data Types

3.1.1 Devices

PyTorch supports both CPU and GPU computation:

```
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
tensor = torch.randn(3, 3).to(device)
```

Always ensure your model and data are on the same device to avoid runtime errors.

3.1.2 Data Types

Common dtypes include `torch.float32` (default), `torch.float64`, `torch.int64`, etc. Specify dtype during creation:

```
x = torch.tensor([1.0, 2.0], dtype=torch.float32)
```

3.2 Datasets and DataLoaders

3.2.1 Dataset Class

A custom dataset inherits from `torch.utils.data.Dataset` and implements:

```
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, data, labels):
        self.data = data
        self.labels = labels

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        return self.data[idx], self.labels[idx]
```

3.2.2 DataLoader

The `DataLoader` handles batching, shuffling, and parallel loading:

```
from torch.utils.data import DataLoader

dataloader = DataLoader(
    dataset,
    batch_size=32,
    shuffle=True,
    num_workers=2  # Parallel data loading
)
```

Key parameters:

- `batch_size`: Number of samples per batch
- `shuffle`: Randomize order each epoch
- `num_workers`: Subprocesses for data loading

3.3 Loss Functions

Loss functions quantify prediction error. Common choices:

- **Mean Squared Error (Regression):**

$$\mathcal{L}_{\text{MSE}} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2 \tag{8}$$

```
criterion = nn.MSELoss()
```

- **Cross-Entropy (Classification):**

$$\mathcal{L}_{\text{CE}} = -\sum_{i=1}^{C} y_i \log(\hat{y}_i) \tag{9}$$

```
criterion = nn.CrossEntropyLoss()
```

Note: `CrossEntropyLoss` expects raw logits (no softmax applied).

3.4 Optimizers

Optimizers update model parameters using gradients. From `torch.optim`:

- **SGD (Stochastic Gradient Descent):**

$$\theta_{t+1} = \theta_t - \eta \nabla_\theta \mathcal{L} \tag{10}$$

- **Adam:** Adaptive learning rates with momentum

```
optimizer = torch.optim.Adam(model.parameters(), lr=0.001)
```

3.5 Training Loop Anatomy

A typical training iteration follows this sequence:

1. **Forward Pass:** Compute predictions
   ```
   outputs = model(inputs)
   ```
2. **Compute Loss:** Measure error
   ```
   loss = criterion(outputs, labels)
   ```
3. **Zero Gradients:** Clear previous gradients
   ```
   optimizer.zero_grad()
   ```
4. **Backward Pass:** Compute gradients via backpropagation
   ```
   loss.backward()
   ```
5. **Update Weights:** Apply gradient descent step
   ```
   optimizer.step()
   ```

Critical points:

- Always call `zero_grad()` to prevent gradient accumulation
- Ensure model and inputs are on the same device
- Use `model.train()` for training and `model.eval()` for evaluation

3.6 Model Persistence

Save and load model weights:

```
# Save
torch.save(model.state_dict(), 'model_weights.pth')

# Load
model.load_state_dict(torch.load('model_weights.pth'))
model.eval()  # Set to evaluation mode
```

For complete checkpoints (optimizer state, epoch, etc.):

```
torch.save({
    'epoch': epoch,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'loss': loss,
}, 'checkpoint.pth')
```

4 Building Neural Networks

4.1 The `nn.Module` Class

All neural networks inherit from `torch.nn.Module`. You must define:

- `__init__()`: Initialize layers
- `forward()`: Define forward pass computation

4.2 Common Layers

- **Linear (Fully Connected):** $\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b}$

  ```
  nn.Linear(in_features, out_features)
  ```

- **Activation Functions:**
  - ReLU: $f(x) = \max(0, x)$
  - Sigmoid: $f(x) = \frac{1}{1+e^{-x}}$
  - Tanh: $f(x) = \tanh(x)$

- **Dropout:** Regularization by randomly zeroing activations

  ```
  nn.Dropout(p=0.5)  # 50% dropout rate
  ```

4.3 Example Architecture

A simple feedforward network for tabular data:

```
import torch.nn as nn

class SimpleNN(nn.Module):
    def __init__(self, input_dim, hidden_dim, output_dim):
        super(SimpleNN, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.relu = nn.ReLU()
        self.dropout = nn.Dropout(0.3)
        self.fc2 = nn.Linear(hidden_dim, output_dim)

    def forward(self, x):
```

```
        x = self.fc1(x)
        x = self.relu(x)
        x = self.dropout(x)
        x = self.fc2(x)
        return x
```

Design considerations:

- **Input dimension:** Number of features in your data
- **Hidden dimension:** Typically 32–512, depends on problem complexity
- **Output dimension:** Number of classes (classification) or 1 (regression)

4.4 Sequential Alternative

For simple architectures, use `nn.Sequential`:

```
model = nn.Sequential(
    nn.Linear(input_dim, 128),
    nn.ReLU(),
    nn.Dropout(0.3),
    nn.Linear(128, output_dim)
)
```

5 Debugging and Best Practices

5.1 Common Errors

- **Shape mismatch:** Check dimensions using `.shape`
- **Device mismatch:** Ensure model and data are on same device
- **Gradient explosion/vanishing:** Use gradient clipping or batch normalization
- **No convergence:** Try different learning rates, architectures

5.2 Debugging Checklist

1. Print tensor shapes at each layer
2. Check for NaN values: `torch.isnan(tensor).any()`
3. Verify gradients: `[p.grad for p in model.parameters()]`
4. Overfit on small batch to ensure model can learn
5. Monitor train vs validation loss (detect overfitting)

5.3 Best Practices

- Use `model.train()` and `model.eval()` appropriately
- Wrap evaluation in `torch.no_grad()` to save memory
- Use tqdm for progress bars
- Set random seeds for reproducibility:

```
torch.manual_seed(42)
```

- Save checkpoints periodically during long training runs

6 Further Resources

6.1 Official Documentation

- PyTorch Documentation

- Tensor Operations
- Neural Network Modules
- Optimization Algorithms

6.2 Tutorials

- PyTorch 60-Minute Blitz
- Learn the Basics
- Deep Learning with PyTorch

6.3 Community and Practice

- PyTorch Forums
- Papers with Code – Implementations
- PyTorch Examples Repository

6.4 Advanced Topics

- Learning rate scheduling: `torch.optim.lr_scheduler`
- Custom loss functions and layers
- Mixed precision training with `torch.cuda.amp`
- Distributed training with `torch.nn.parallel`

7 Conclusion

This tutorial covered the essential components of PyTorch programming. Practice these concepts by experimenting with different architectures, hyperparameters, and datasets. Deep learning is an empirical field; so hands-on experience is crucial for mastery.

Good luck!